Operating Systems Technologies

University of Minho

Guide 3
eBPF

2026

Goal

This guide will allow you to experiment with writing eBPF programs, instrumenting different types of probes (tracepoints and kprobes), and observing operations across multiple OS layers: the system call interface, virtual file system (VFS), EXT4 file system, and block I/O layer. You will also work with different eBPF maps (ring buffer, hash map) and explore associated helper function calls.

Tools / Technologies. We will use eBPF and the `libbpf` library to implement and interact with the eBPF programs.

Learning Outcomes. Gain practical experience with eBPF and the `libbpf` library. Experiment different kernel instrumentation points. Experiment event capturing at multiple OS layers.

1 Warmup

Start by installing the required packages:

```
$ sudo apt install libbpf-dev clang llvm auditd
```

2 Exercises

Implement a program that uses eBPF to trace system calls. Use the provided base code composed of:

- *systracer.bpf.c* – The eBPF program that runs inside the kernel. It instruments the sys_enter tracepoint to capture all syscall-related events and forwards relevant information to the user space via a ring buffer.
- *systracer.c* – The user-space component of the program. It is responsible for loading the eBPF program, attaching it to the appropriate kernel hooks, handling events received from the kernel (via a ring buffer), and presenting the collected syscall data to the user.
- *systracer.h* – A shared header file containing data structures and definitions used by both the user-space program and the eBPF program.
- *Makefile* – Builds the entire project. It compiles the eBPF source into bytecode (*systracer.bpf.o*), generates the libbpf skeleton header (*systracer.skel.h*), and extracts kernel type definitions into *vmlinux.h* (via bpftool). It also creates a header file mapping syscall numbers to syscall names (*syscalls_table.h*).

Useful documentation:

- eBPF: `https://ebpf.io/what-is-ebpf/`, `https://docs.ebpf.io`.
- Libbpf: `https://libbpf.readthedocs.io/en/latest/libbpf_overview.html`

Inspect the source code, then build and execute the `systracer` program:

```
$ makefile
$ sudo ./systracer
```

2.1 Tracing Specific Syscalls

Use the provided program `file_rw_multi` and analyze its behavior with `systracer`. Inspect its code, and start by running it with the argument *rw* (*i.e.*, mix of read and write operations without O_DIRECT)

The base version of `systracer` traces all system calls executed by all processes in the system (except for `systracer` itself — see the filtering logic in `systracer.bpf.c`, lines 25–30).
Tracing all system calls may generate excessive and unnecessary data. Modify the program so that only the following system calls are traced: `openat`, `read`, `write`, and `close`.

1. Start by implementing an auxiliary function that encapsulates the common logic. For example, implement a function that receives the PID and the syscall ID as arguments, constructs an event of type `struct data_t`, and submits it to user space through the ring buffer:

   ```
   int create_and_submit_event(u32 pid, u64 id);
   ```

2. Now, instead of attaching the eBPF program to the generic `sys_enter` tracepoint, use the dedicated tracepoints corresponding to each of the specified system calls.

   Inspect the format of each tracepoint using:

   ```
   $ sudo cat /sys/kernel/tracing/events/syscalls/sys_enter_<syscall>/format
   ```

   Use the `BPF_PROG` macro, which allows you to define a tracepoint (or raw tracepoint) eBPF program with explicitly declared, typed arguments. It automatically maps the tracepoint context to the function parameters, so you can access the tracepoint fields directly instead of manually extracting them from a context structure. More info at: `https://docs.ebpf.io/ebpf-library/libbpf/ebpf/BPF_PROG/`.

   For example:

   ```
   SEC("tp/syscalls/sys_enter_openat")
   int BPF_PROG(sys_enter_openat,
                struct pt_regs *regs,
                long syscall_id,
                int dfd,
                const char *filename)
   {
       ...
   }
   ```

   Use the auxiliary function `create_and_submit_event` to avoid duplicating the event-creation and submission logic across tracepoint handlers.

2.2 Filtering by Process ID

Tracing these four operations from every process can still generate an overwhelming number of events, especially when we are only interested in tracing a single process. Modify the program so that it captures only system calls executed by a specific PID, which is passed as a command-line argument to `main`.

1. Use an eBPF map (e.g., `BPF_MAP_TYPE_HASH`) to share the target PID from the user-space program to the eBPF program. From user space, use the `bpf_map__update_elem` function to insert the PID into the map before attaching the eBPF programs to the target probes (i.e., before calling `systracer_bpf__attach`). More info at:
   - `https://docs.ebpf.io/linux/map-type/BPF_MAP_TYPE_HASH/`
   - `fhttps://docs.ebpf.io/ebpf-library/libbpf/userspace/bpf_map__update_elem/`
2. At the eBPF program, implement an auxiliary function to encapsulate the PID filtering logic. This function should determine whether a given PID should be ignored or traced:

   ```
   int to_discard(u32 pid);   // 1 → discard, 0 → collect
   ```

   The tracepoint handlers should invoke this function before creating and submitting an event, ensuring that the filtering logic is centralized and not duplicated across multiple tracepoints.

2.3 Tracing operations at different levels

Let us see what is happening at the lower levels, such as at the Virtual File System (VFS), the FS (ext4 in this case), and the block I/O layer.

1. Modify `struct data_t` to include a new field (`int type`) that identifies the layer at which the operation is being traced:
   - 0 – System call layer
   - 1 – VFS layer
   - 2 – File system layer (e.g., EXT4)
   - 3 – Block I/O (BIO) layer

   Update the `create_and_submit_event` function accordingly so that it receives the layer type as an argument and stores it in the event structure before submitting it to the user space.
2. The following example uses a kprobe to instrument the `vfs_open` function at the VFS layer:

   ```
   SEC("kprobe/vfs_open")
   int BPF_KPROBE(vfs_enter_open,
                  const struct path *path,
                  struct file *file)
   {
       u32 pid = bpf_get_current_pid_tgid() >> 32;
       if (to_discard(pid)) return 0;

       return create_and_submit_event(pid, MYOP_OPENAT, 1);
   }
   ```

   The `BPF_KPROBE` macro allows you to define a kprobe handler with typed function arguments that match the probed kernel function's prototype. It automatically retrieves the parameters from `struct pt_regs` and makes them directly accessible inside the eBPF program, improving readability and safety. More info at: `https://docs.ebpf.io/ebpf-library/libbpf/ebpf/BPF_KPROBE/`

Based on this example, use kprobes to instrument the `vfs_read` and `vfs_write` functions to trace read and write operations at the VFS layer.

3. Follow a similar approach to instrument the `ext4_file_read_iter` and `ext4_file_write_iter` functions from the EXT4 file system layer.

   Example for the open operation:

   ```
   SEC("kprobe/ext4_file_open")
   int BPF_KPROBE(ext4_enter_open,
                  struct inode *inode,
                  struct file *filp)
   {
       u32 pid = bpf_get_current_pid_tgid() >> 32;

       if (to_discard(pid))
           return 0;

       return create_and_submit_event(pid, MYOP_OPENAT, 2);
   }
   ```

   Hint: Use the site `https://elixir.bootlin.com` to inspect these function headers and check their formats (arguments and return values).

4. Use the tracepoint for the `block_rq_issue` event to trace block I/O requests issued to the disk.

   Inspect the tracepoint format with:

   ```
   $ sudo cat /sys/kernel/tracing/events/block/block_rq_issue/format
   ```

   Add the following struct to *systracer.bpf.c* and use it to access the function arguments.

   ```
   struct block_rq_submission {
       uint16_t common_type;
       uint8_t common_flags;
       uint8_t common_preempt_count;
       int32_t common_pid;

       uint32_t dev;
       uint64_t sector;
       uint32_t nr_sectors;
       uint32_t nr_bytes;
       uint16_t ioprio;
       int8_t rwbs[10];
       int8_t comm[16];
       int8_t cmd[4];
   };
   ```

   Use the `rwbs` field to distinguish between read and write operations. For example:

   ```
   if (ctx->rwbs[0] == 'R') {
       // handle read request
   ```

```
} else if (ctx->rwbs[0] == 'W') {
    // handle write request
}
```

Based on the operation type, generate and submit the appropriate event.

5. Compare the results in the following scenarios:

   - Running the `file_rw_multi` program without `O_DIRECT` (argument: `rw`).
   - Running the `file_rw_multi` program with `O_DIRECT` (argument: `rwo`).

   Observe how the system calls, VFS operations, file system layer events, and block I/O requests differ between these modes.

   Question: Why do you only observe block-layer requests when O_DIRECT is active?

3 Extras

Extend the program so that any new processes created by the traced process are also traced.
Use the `sched_process_fork` tracepoint, which is triggered whenever a process forks and creates a new child process. Inspect the tracepoint format using:

```
$ sudo cat /sys/kernel/tracing/events/sched/sched_process_fork/format
```